Documentación Técnica
Sistema de Análisis LTV para Pacifiko.com

Luis Cordón

29 de Abril del 2026

Contents

1 Conceptos clave del sistema **3**

2 Arquitectura y organización del proyecto **3**
2.1 Estructura de directorios (detallada) . . . 3
2.2 Dependencias principales y su propósito . . . 5

3 Flujo general del programa (paso a paso técnico) **5**

4 Cálculos relevantes (fórmulas y algoritmos) **6**
4.1 Contribution profit por orden (DataRepository) . . . 6
4.2 Neteo de IVA . . . 6
4.3 Tipo de cambio (FXEngine) . . . 6
4.4 Cohortes trimestrales relativas . . . 7
4.5 Supuestos financieros . . . 7
4.6 Retención (costos de marketing recurrente) . . . 7

5 Sistemas internos relevantes **7**
5.1 Sistema de scoring (MetricsQualityAnalyzer) . . . 7
5.2 Transformación de granularidad (TimeGranularityAdapter) . . . 8
5.3 Generación de gráficos (CategoryVisualizer) . . . 8
5.4 Ordenamiento global persistente . . . 8

6 Estructura del código por capas (detalle de clases críticas) **8**
6.1 DataRepository . . . 8
6.1.1 QueryEngine . . . 8
6.1.2 DataLoader . . . 8
6.1.3 DataValidator . . . 9
6.1.4 DataMerger . . . 9
6.2 Model . . . 9
6.2.1 Order . . . 9
6.2.2 Customer . . . 9
6.2.3 LTVController . . . 9
6.3 Category . . . 9
6.3.1 BaseDimensionOrchestrator (abstracto) . . . 9
6.3.2 GlobalLTVOrchestrator . . . 10
6.3.3 DimensionQueryEngine . . . 10

7 Manejo de datos y credenciales **10**
7.1 Fuentes de datos . . . 10
7.2 Credenciales y seguridad . . . 10
7.3 Inputs y outputs . . . 10

8 Extensibilidad y puntos de modificación **11**
8.1 Dónde modificar sin romper el sistema . . . 11
8.2 Extensiones posibles sin refactorización mayor . . . 11

9 Errores comunes y su resolución (consejos técnicos) **11**
9.1 DataRepository . . . 11
9.2 Model . . . 11
9.3 Category / Multi-dimensión . . . 12
9.4 Menús y autenticación . . . 12

10 Uso de IA para debugging (recomendaciones) **12**
10.1 Cómo describir un problema para obtener ayuda efectiva . . . 12
10.2 Ejemplo de prompt efectivo . . . 12
10.3 Consideraciones de seguridad al compartir información . . . 13

11 Soporte y contacto técnico **13**

12 Anexos: Ejemplos de configuración rápida **13**
12.1 Ejemplo de archivo SUPUESTOS.xlsx para Guatemala (hoja `1PGT`) . . . 13
12.2 Ejemplo de archivo CAC.xlsx (hoja `GT`) . . . 13
12.3 Ejemplo de comando SSH para túnel (Linux/Mac) . . . 13

1 Conceptos clave del sistema

- **LTV (Lifetime Value).** Contribution profit acumulado de un cliente. Se calcula restando costos variables (envío, comisiones, fraude, infraestructura, COGS, retención) del revenue, sumando SOIS e ingresos por envío. Existen dos versiones: **LTV bruto** (sin descontar CAC) y **LTV neto** (LTV bruto - CAC).
- **Cohorte.** Grupo de clientes cuya primera compra ocurre en el mismo período. El sistema maneja cohortes trimestrales de forma nativa (Q1, Q2, ...) y transforma a otras granularidades (mensual, semanal, semestral, anual) mediante `TimeGranularityAdapter`.
- **Frecuencia / Recurrencia.** Número de compras únicas por cliente. Se analiza por cohorte y dimensión.
- **CAC (Costo de Adquisición).** Gasto en marketing por cliente, asignado por cohorte. Se lee desde `CAC.xlsx` o desde la columna `cac` en `SUPUESTOS.xlsx`.
- **Business Units (BU):** 1P (inventario propio), 3P (marketplace), FBP (fulfillment), TM (manufactura), DS (dropshipping). Cada una tiene supuestos financieros separados en hojas Excel (ej. `1PGT`, `3PT`, etc.).
- **SOIS (Stock On Inventory System).** Valor de inventario por producto, utilizado como componente positivo en el contribution profit.

2 Arquitectura y organización del proyecto

2.1 Estructura de directorios (detallada)

```
Project Root/
 DataRepository/                  # Extracción y transformación (ETL)
    Data/                        # Conexión y carga
       query_engine.py          # MySQL con parámetros por país
       data_loader.py           # Orquesta BD + Excels
       data_validator.py        # Limpieza, rescate de PIDs
       data_merger.py           # Joins con catálogo y SOIS
    Processing/                  # Lógica de negocio
       cohort_builder.py        # Cohortes relativas (Q1, Q2...)
       retention_applier.py     # Distribuye costo retención
       assumption_applier.py    # Supuestos por BU/cohorte
       metrics_calculator.py    # CP, FX, IVA
    Output/                      # Resultados intermedios
        final_dataset_builder.py # Estandarización de columnas
        data_exporter.py         # CSV/Excel + informe TXT
 Model/                           # Dominio y análisis LTV
    Domain/                      # Objetos de negocio
       order.py                 # Atributos y calculate_cp()
       customer.py              # Agrega órdenes, LTV, segmentación
       controller.py            # LTVController: construye objetos
    Analytics/                   # Métricas y análisis
       cohort_analyzer.py       # Matriz LTV por cohorte
       unit_economics.py        # LTV neto, ROI, payback
       category_value_analyzer.py # Correlación dimensión vs LTV
       dashboard_analyzer.py    # Resúmenes ejecutivos
    Data/                        # Lectura de datos reales
       real_data_repository.py # Carga CSV desde Data_LTV
       cac_repository.py        # CAC por cohorte
    Output/                      # Reportes finales
```

```
      data_exporter.py        # Excel multi-hoja
Category/                      # Análisis multi-dimensión
   Grouping/                   # Agrupadores
      category_grouper.py     # Behavioral (filtrado)
      entry_grouper.py        # Entry-based (atribución total)
   Cohort/                     # Gestión dinámica de cohortes
      cohort_config.py        # Configuración (granularidad, rango)
      cohort_manager.py       # Fachada para IDs de cohorte
      cohort_grouper.py       # Agrupa customers por cohorte
   Analytics/                  # Analizadores específicos
      frequency_analyzer.py   # % clientes con 2da,3ra,4ta compra
      time_analyzer.py        # Mediana días entre compras
      conversion_analyzer.py  # Tasas de conversión (30d,60d,...)
      metrics_analyzer.py     # Scoring con penalización
      percentile_scoring.py   # Wrapper de scoring
   Orchestrators/              # Orquestación
      base_dimension_orchestrator.py # Template method
      behavior_orchestrator.py # Ejemplo para categoría
      global_orchestrator.py   # Ejecuta todas las dimensiones
   Reporting/                  # Exportación
      base_exporter.py        # Genera resúmenes y scoring
      product_exporter.py     # Añade columna PID
      category_visualizer.py  # Gráficos (matplotlib/seaborn)
      dashboard_exporter.py   # TXT top/bottom rankings
   Utils/                      # Utilidades
       dimension_config.py     # Configuración por dimensión
       utils_ue.py             # DataFrames de Unit Economics
Run/                           # Configuración, autenticación, menús
   Config/                     # Rutas, credenciales, modos
      paths.py                # Directorios de trabajo
      credentials.py          # Credenciales híbridas (JSON+CSV)
      credential_store.py     # Cifrado Fernet
      vault_manager.py        # Legacy CSV (deprecated)
      dev_mode_manager.py     # Modo desarrollador (auto-login)
      config_loader.py        # JSON genérico
   Security/                   # Autenticación y gestión de usuarios
      auth_service.py         # Fachada de autenticación
      user_manager.py         # Usuarios, credenciales por país
      credential_manager.py   # Configuración global por país
   Country/                    # Multi-país
      country_context.py      # Datos del país (start_year, moneda)
      country_loader.py       # Carga desde JSON
      country_selector.py     # Menú interactivo
   FX/                         # Tipo de cambio
      fx_engine.py            # Tasas por cohorte + fallback
   Services/                   # Servicios auxiliares
      script_runner.py        # Ejecuta scripts externos
      ssh_service.py          # Túnel SSH (legacy)
      ssh_manager.py          # Gestión centralizada SSH
      cohort_supuestos_manager.py # CRUD de supuestos
      time_granularity_adapter.py # Transformación quarterly → otras
```

```
      auxiliary_exporter.py   # Backups en AUXILIAR
      system_cleaner.py       # Limpieza de logs/cache
      credential_migration.py # Verificador migración CSV→JSON
   Core/                       # Componentes centrales
      cohort_context_manager.py # Unifica supuestos por cohorte
      ssh_manager.py          # Singleton SSH
   Utils/                      # Utilidades varias
      logger.py               # Logger dual (consola+archivo)
      retry.py                # Decorador de reintentos
      get_flexible_input.py   # Entrada sí/no flexible
   Menu/                       # Menús interactivos
       menu_auth.py            # Login, crear usuario, recuperar pass
       menu_config.py          # Configuraciones (modos, granularidad)
       menu_executor.py        # Ejecución de pipelines
       menu_controller.py      # Orquestador principal
main.py                         # Punto de entrada
```

2.2 Dependencias principales y su propósito

- `pandas, numpy` → manipulación de datos y cálculos vectorizados.
- `sqlalchemy, pymysql` → conexión a MySQL y ejecución de queries.
- `openpyxl` → lectura/escritura de archivos Excel (mantiene formato).
- `bcrypt, cryptography` → hash de contraseñas de usuario y cifrado de credenciales DB.
- `matplotlib, seaborn` → generación de gráficos de dispersión y tendencias.
- `tkinter` → diálogos nativos para selección de carpetas.
- `termios, tty, msvcrt` → entrada de contraseña con asteriscos (cross-platform).

3 Flujo general del programa (paso a paso técnico)

1. **Autenticación** (`MenuAuth`) → login o creación de usuario. Las credenciales se almacenan cifradas (Fernet) en `Run/Config/secure/credentials.enc`. La clave está en `.key`.
2. **Selección de país** → carga configuración (año inicio, moneda, hojas Excel) desde JSON en `Run/Country/definitions/`.
3. **Menú principal** → permite ejecutar pipelines, configurar, usar buscador.
4. **Pipeline DataRepository** (`mainDR.py`):
   - `QueryEngine` establece conexión MySQL (con túnel SSH opcional).
   - Ejecuta query parametrizada por país (GT sin IVA, CR con IVA neteado en SQL).
   - `DataLoader` carga órdenes, SOIS.xlsx, SUPUESTOS.xlsx, catalogLTV.xlsx.
   - `DataValidator` limpia: asigna `PID-DESCONOCIDO`, filas con price/quantity nulos.
   - `DataMerger` une catálogo (b_unit, categorías) y SOIS (por vigencia).
   - `CohortBuilder` genera cohorte relativa (Q1, Q2...) usando año base del país.
   - `RetentionApplier` distribuye costos de retención usando fallback dinámico.
   - `AssumptionApplier` asigna supuestos financieros por BU y cohorte.
   - `MetricsCalculator` aplica tipo de cambio (FXEngine), netea IVA (solo GT), calcula revenue, base_cost y contribution profit.
   - `FinalDatasetBuilder` estandariza columnas y tipos.
   - `DataExporter` guarda como CSV y genera informe TXT.
5. **Pipeline Model** (`mainMD.py`):
   - `RealDataRepository` lee CSV generado por DataRepository (el más reciente).
   - `LTVController` transforma filas en objetos `Order` y `Customer`.
   - `CohortAnalyzer` construye matriz LTV por cohorte (sin forward fill).

- `UnitEconomicsAnalyzer` integra CAC, calcula LTV neto, ROI, payback period.
- `CategoryValueAnalyzer` correlaciona dimensión de entrada (categoría/marca) con LTV.
- `DashboardAnalyzer` produce resúmenes ejecutivos y segmentación por percentiles.
- `GlobalLTVOrchestrator` (si se invoca) ejecuta análisis para todas las dimensiones configuradas (1..6).
- `DataExporter` genera Excel múltiples hojas y TXT.

4 Cálculos relevantes (fórmulas y algoritmos)

4.1 Contribution profit por orden (DataRepository)

Fórmula completa (a nivel de ítem de orden):

$$\begin{aligned}\mathrm{CP}_{\mathrm{item}} = \ & \mathrm{revenue} - \mathrm{item_cost} + \mathrm{sois} \\ & + (\mathrm{shipping_cost} + \mathrm{shipping_revenue}) \\ & + (\mathrm{credit_card_cost} + \mathrm{cod_cost}) \\ & + \mathrm{fc_variable_headcount} + \mathrm{cs_variable_headcount} \\ & + \mathrm{fraud_cost} + \mathrm{infrastructure_cost} \\ & - \mathrm{retention_cost}\end{aligned}$$

Donde:

- `revenue = quantity * price_usd`
- `item_cost` es el costo del producto en USD (neto de IVA).
- `sois` es el valor de inventario (positivo).
- Los costos de envío y headcount son negativos en la base de datos (se suman directamente).
- `retention_cost` ya está distribuido (es positivo, por eso se resta).

4.2 Neteo de IVA

- **Guatemala:** Los datos de la BD vienen con IVA incluido. Se aplica:

$$\mathrm{price}_{\mathrm{neto}} = \frac{\mathrm{price}_{\mathrm{bruto}}}{1.12}$$

- **Costa Rica:** La query SQL ya aplica el neteo usando la tabla `pac_cabys`.

4.3 Tipo de cambio (FXEngine)

- Se lee archivo `TIPO_DE_CAMBIO.xlsx` con columna `cohort` (formato Q1, Q2... o YYYY-MM) y columna de tasa (detectada automáticamente).
- Algoritmo de fallback:
  1. Búsqueda exacta de cohorte.
  2. Para granularidades no quarterly, mapeo (ej. YYYY-MM → trimestre).
  3. Si no hay coincidencia, extrae número de cohorte (Q1→1, 2024-03→202403) y busca la tasa más cercana por diferencia absoluta.
  4. Último recurso: tasa por defecto del país (GTQ 7.66, CRC 550).
- La conversión a USD se hace en `MetricsCalculator`:

$$\mathrm{USD} = \frac{\mathrm{moneda_local}}{\mathrm{tasa}}$$

4.4 Cohortes trimestrales relativas

- Año base: Guatemala = 2020, Costa Rica = 2022.
- Fórmula para cohorte QN:

$$\text{año_real} = \text{base_year} + \left\lfloor \frac{N-1}{4} \right\rfloor$$

- Ejemplo: base_year=2020, Q5 → 2020 + (5-1)//4 = 2021.
- Se usa también para filtrar cohortes anteriores al inicio del país.

4.5 Supuestos financieros

Se leen desde `SUPUESTOS.xlsx`. Columnas clave:
- `cogs` (COGS como porcentaje del revenue) → solo aplica a BU `TM`.
- `retention` (costo fijo por orden recurrente).
- `cac` (costo de adquisición por cliente).
- `shipping_cost`, `shipping_revenue` (valores fijos por orden).
- `credit_card_payment`, `cash_on_delivery_comision`, `fraud`, `infrastructure` → porcentajes del revenue.
- `fc_variable_headcount`, `cs_variable_headcount` → costos fijos por orden.

4.6 Retención (costos de marketing recurrente)

- Se lee desde SUPUESTOS.xlsx (columna `retention`), en formato trimestral relativo.
- `RetentionApplier` distribuye el costo por cliente recurrente:

$$\text{costo_retención_item} = \frac{\text{budget_retención_cohorte}}{\text{n_órdenes_recurrentes_del_cliente} \times \text{n_items_en_la_orden}}$$

- Solo se asigna a órdenes que no son la primera compra del cliente (recurrentes).

5 Sistemas internos relevantes

5.1 Sistema de scoring (MetricsQualityAnalyzer)

Objetivo: penalizar dimensiones con muestra pequeña y normalizar puntuaciones.
- **Umbral mínimo de confianza:** `MIN_CONFIDENCE_CLIENTS = 100` (configurable).
- **Penalización por muestra pequeña:**

$$\text{penalty} = \begin{cases} \left(\frac{n}{\text{umbral}}\right)^2 & \text{si } n < \text{umbral} \\ 1 & \text{en otro caso} \end{cases}$$

- **Confidence Score:** percentil lineal del tamaño de muestra respecto al máximo.

$$\text{confidence} = \frac{n - \min(n)}{\max(n) - \min(n)}$$

- **Performance Score:** promedio ponderado de:
  - Frequency Score (peso 0.2)
  - Time Score (peso 0.3)
  - Conversion Score (peso 0.1)
  - LTV Score (peso 0.4)
- **Final Score:**

$$\text{final} = \text{performance} \times \sqrt[3]{\text{confidence}}$$

- Las etiquetas cualitativas: Excelente (0.8), Bueno (0.6), Regular (0.4), Bajo (0.2), Crítico (<0.2).

5.2 Transformación de granularidad (TimeGranularityAdapter)

- Convierte supuestos trimestrales (Q1, Q2...) a:
  - **Mensual:** divide retención entre 3, COGS se mantiene.
  - **Semanal:** divide entre 13 (semanas por trimestre).
  - **Semestral:** suma de dos trimestres consecutivos.
  - **Anual:** suma de cuatro trimestres.
- El año base se obtiene de `CountryContext.cohort_start_year`.
- Para cohortes absolutas (YYYY-MM, YYYY-Wxx, YYYY-H1), se parsea con regex.

5.3 Generación de gráficos (CategoryVisualizer)

- Usa `matplotlib` y `seaborn`.
- Tipos de gráficos:
  - Dispersión: métrica de comportamiento (frecuencia, tiempo, conversión) vs AOV y vs Total_Clientes.
  - LTV vs comportamiento.
  - LTV vs Final Score, con puntos coloreados por bucket de AOV (LOW_TICKET, MID_TICKET, HIGH_TICKET).
- Se guardan en subcarpeta `Plots_Dimensión_timestamp/` dentro del directorio de resultados.

5.4 Ordenamiento global persistente

- Guardado en `config/user_sort_preference.json` vía `ConfigLoader`.
- Opciones:
  - `clients` → ordenar por número de clientes (default)
  - `ltv` → ordenar por LTV promedio
  - `score` → ordenar por Final Score
- Afecta al buscador (`DimensionQueryEngine`) y a rankings en dashboards.
- Se puede cambiar interactivamente en el menú de configuraciones.

6 Estructura del código por capas (detalle de clases críticas)

6.1 DataRepository

6.1.1 QueryEngine

- Atributos: `user, password, host, db, country_code`
- Métodos: `__init__`, `fetch_orders(start_date, end_date)`
- Queries: `QUERY_GT` (sin neteo de IVA) y `QUERY_CR` (con neteo usando tax_rate).
- Parámetros de fechas: start_date por defecto según país (2020-01-01 GT, 2022-01-01 CR); end_date por defecto hoy - 7 días.

6.1.2 DataLoader

- Coordina `load_all_sources()`: ejecuta `fetch_orders` y carga Excel.
- Utiliza `CountryContext` para obtener nombre de hojas (ej. "GT", "CR").
- Para SUPUESTOS, busca hojas con formato {`BU`}{`código país`} (ej. `1PGT`).
- Realiza backup automático de órdenes crudas en CSV.

6.1.3 DataValidator

- Renombra columnas a nomenclatura interna.
- Rescate de PIDs nulos → asigna `"PID-DESCONOCIDO"`.
- Elimina filas con `quantity` o `price` nulos.
- Convierte SOIS de QTZ a USD usando tipo de cambio fijo (7.66) si es necesario.

6.1.4 DataMerger

- `merge_catalog`: left join con catálogo (elimina duplicados por PID). Asigna `b_unit`, `category`, `subcategory`, `brand`, `name`. Rellena valores faltantes con "OTROS" / "UNKNOWN".
- `merge_sois`: aplica SOIS con validación de vigencia (fecha entre `fecha_inicio` y `fecha_fin`). Maneja PIDs inválidos (set "NONE", "null") asignando SOIS = 0.

6.2 Model

6.2.1 Order

- Atributos: `order_id`, `customer_id`, `order_date`, `quantity`, `revenue`, `cost`, `sois`, `shipping_cost`, `shipping_revenue`, `credit_card_cost`, `cod_cost`, `fc_variable`, `cs_variable`, `fraud_cost`, `infrastructure_cost`, `retention_cost`, `category`, `subcategory`, `brand`, `name`, `business_unit`.
- Método `calculate_cp()` implementa la fórmula del contribution profit.
- Propiedad `subcategory_brand` (read-only) para análisis combinado.

6.2.2 Customer

- Almacena lista de `Orders`.
- Cachés: `_timeline_cache`, `_unique_orders_cache`.
- Métodos:
  - `total_revenue()`, `total_cp()`
  - `total_orders()` → número de compras únicas.
  - `ltv_timeline()` → evolución del LTV acumulado.
  - `get_entry_dimension(mode)` → categoría/subcategoría/marca/producto de primera compra.
  - `get_cohort_id(granularity)` → usa `CustomerCohortCache` para evitar recalcular.

6.2.3 LTVController

- `process_raw_data(raw_data_list)`: convierte diccionarios en objetos `Order` y los asigna a `Customer`.
- Deduplicación usando `fingerprint = f"order_id_revenue_category"` para evitar duplicados.
- Maneja errores de fechas y cantidad nula (setea quantity = 1.0).

6.3 Category

6.3.1 BaseDimensionOrchestrator (abstracto)

- Método template `run()`:
  1. Agrupa customers por dimensión usando `EntryBasedBehaviorGrouper` o `CategoryGrouper`.
  2. Para cada grupo, calcula métricas históricas (sin cohorte) y por cohorte (via `CohortGrouper`).
  3. Aplica scoring con `MetricsQualityAnalyzer`.
  4. Redistribuye resultados en diccionario {`frequency, time, conversion, metadata`}.
- Clases concretas: `CategoryBehaviorOrchestrator`, `SubcategoryBehaviorOrchestrator`, etc.

6.3.2 GlobalLTVOrchestrator

- Recibe lista de dimensiones a procesar (1..6) y `grouping_mode`.
- Para cada dimensión, instancia el orquestador específico y el exporter.
- Genera carpetas de salida con timestamp y sufijos de modo y granularidad.
- Opcional: aplica filtro por dimensiones (category, subcategory, brand) antes de procesar.

6.3.3 DimensionQueryEngine

- Motor interactivo para buscar dimensiones específicas.
- Métodos: `unified_search()`, `query()`, `interactive_search()`.
- Soporta búsqueda por PID exacto para productos.
- Utiliza `CohortManager` para granularidad dinámica.
- Ordenamiento global aplicado según preferencia del usuario.

7 Manejo de datos y credenciales

7.1 Fuentes de datos

- **MySQL:** tablas `oc_order`, `oc_order_product`, `oc_order_history`, `oc_product`, `oc_purpletree_vendor_com`
- **Excel:** `SOIS.xlsx`, `SUPUESTOS.xlsx`, `catalogLTV.xlsx`, `CAC.xlsx`, `TIPO_DE_CAMBIO.xlsx`.

7.2 Credenciales y seguridad

- **Cifrado:** Fernet (AES-128 CBC + HMAC). Los archivos se guardan en `Run/Config/secure/credentials.e`
- **Clave:** `secure/.key` (generada automáticamente al primer uso).
- **Migración:** Si existe `data_xlsx/credentials_vault.csv`, se migra automáticamente a JSON cifrado (dual-write).
- **Usuarios del sistema:** Almacenados en `Run/Config/data_json/credentials.json` (hash bcrypt).
- **Credenciales por país:** Cada usuario puede tener diferentes credenciales DB para GT y CR (en `db_by_country`).

7.3 Inputs y outputs

- **Inputs:** carpeta configurable (default `data_xlsx/`).
- **Outputs DataRepository:** `Data_LTV_{país}/Resultado_Unit_Economics_*.csv`.
- **Outputs Model:** `Resultados_LTV_{país}`$_{timestamp}/con$:
  - `Analisis_Categorias_LTV_*.xlsx`
  - `Analisis_Productos_LTV_*.xlsx`
  - `Resumen_Ejecutivo_LTV_*.txt`
  - Subcarpeta `Plots_Categorias_timestamp/` (gráficos)

8 Extensibilidad y puntos de modificación

8.1 Dónde modificar sin romper el sistema

- **Agregar un nuevo país:**
  1. Crear archivo JSON en `Run/Country/definitions/` (ej. `mx.json`).
  2. Agregar hoja en `SUPUESTOS.xlsx` con sufijo del país (ej. `1PMX`, `3PMX`) y hoja en `catalogLTV.xlsx` con nombre "MX".
  3. Configurar las credenciales DB para ese país en el menú de creación/edición de usuario.
- **Cambiar la granularidad por defecto:** modificar `MenuConfig.current_granularity` (valor inicial) y la opción en el menú.
- **Ajustar el scoring:**
  - Umbral de confianza: cambiar `MIN_CONFIDENCE_CLIENTS` en `MetricsQualityAnalyzer`.
  - Pesos de las métricas: editar `_calculate_weighted_scores()` en `BaseExporter`.
- **Agregar una nueva dimensión de análisis (por ejemplo "proveedor"):**
  1. Extender `DimensionMode` (enum) y `DIMENSION_CONFIGS` en `dimension_config.py`.
  2. Implementar un orquestador concreto (heredar de `BaseDimensionOrchestrator`) y su exporter (heredar de `BaseExporter`).
  3. Registrar en los mapas `_get_orchestrator_class` y `_get_exporter_class` de `GlobalLTVOrchestrator`.
- **Modificar la fórmula de CP:** editar `Order.calculate_cp()` (Model layer) y `MetricsCalculator.run()` (DataRepository).

8.2 Extensiones posibles sin refactorización mayor

- Nuevos formatos de exportación (Parquet, JSON) → extender `DataExporter` (`export_to_parquet()`).
- Nuevos tipos de gráficos → agregar métodos en `CategoryVisualizer`.
- Nuevos modos de ordenamiento global → añadir a `SORT_OPTIONS` en `MenuConfig` y en `DimensionQueryEngine`
- Transformaciones de granularidad adicionales (ej. "bimensual") → extender `TimeGranularityAdapter`.
- Nuevos analizadores de comportamiento → crear subclase de `BaseDimensionOrchestrator` y agregar al pipeline.

9 Errores comunes y su resolución (consejos técnicos)

9.1 DataRepository

- **Error de conexión a BD:**
  - Verificar credenciales en `CredentialStore` (menú de usuario).
  - Probar túnel SSH manualmente: `ssh -N -L 3336:localhost:3306 usuario@host`.
  - Revisar que los puertos no estén ocupados (`netstat -tulpn | grep 3336`).
- **Hoja de Excel no encontrada:**
  - Verificar nombres: para GT, las hojas de SUPUESTOS deben ser `1PGT`, `3PGT`, etc.
  - SOIS y catalogLTV deben tener hoja con nombre exacto `GT`.
- **PIDs desconocidos masivos:** Revisar que `catalogLTV.xlsx` tenga columna `product_pid` y que las órdenes tengan `product_pid` no nulo.
- **SOIS sin vigencia:** Verificar que `Fecha_inicio` y `Fecha_fin` estén en formato datetime y cubran el rango de órdenes.

9.2 Model

- **Falta CAC:** LTV_neto = LTV_bruto. Para solucionar: agregar `CAC.xlsx` con hoja del país o columna `cac` en SUPUESTOS.
- **No hay datos en Data_LTV:** Asegurar que se ejecutó `mainDR.py` al menos una vez.

- **Ordenamiento de cohortes incorrecto en reportes:** El sistema usa orden lexicográfico por defecto. Si se necesita cronológico, modificar el exporter para usar `CohortUtils.parse_cohort_id`.
- **Fechas mal parseadas:** El CSV debe tener columna `order_date` en formato ISO (YYYY-MM-DD HH:MM:SS). Si hay otros formatos, el `RealDataRepository` fallará.

9.3 Category / Multi-dimensión

- **Dimensiones sin grupos:** Verificar que `catalogLTV.xlsx` tenga cobertura de `category`, `subcategory`, `brand`, `name`. Ejecutar con un país que tenga datos.
- **Penalización alta:** Si muchas dimensiones tienen `n < MIN_CONFIDENCE_CLIENTS`, los scores serán bajos. Ajustar umbral o recolectar más datos.
- **Buscador no encuentra productos:** Ejecutar pipeline completo al menos una vez para poblar `Data_LTV`. El buscador lee el CSV más reciente.

9.4 Menús y autenticación

- **No se puede autenticar:** Verificar que el usuario existe (listar usuarios en menú). Si se olvidó la contraseña, usar recuperación (valida credenciales DB).
- **Fallo en migración de credenciales:** Si `secure/credentials.enc` está corrupto, eliminarlo y reiniciar. Se regenerará con datos del CSV legacy si existe.
- **Credenciales DB no guardadas:** Usar la opción "Editar usuario" en menú de autenticación para agregar credenciales por país.

10 Uso de IA para debugging (recomendaciones)

10.1 Cómo describir un problema para obtener ayuda efectiva

1. **Fragmento del error:** Copiar el traceback completo o el mensaje de error exacto.
2. **Contexto de ejecución:** Indicar qué pipeline se ejecutó (DR, Model, ambos), parámetros (país, granularidad, filtros), y si es la primera ejecución o recurrente.
3. **Estado de los datos:** Mencionar si los archivos Excel existen, si `Data_LTV` tiene datos, etc.
4. **Entorno:** Sistema operativo, versión de Python, si se usa túnel SSH.
5. **Configuraciones:** Modo de agrupación (entry/behavioral), granularidad, modo de conversión.

10.2 Ejemplo de prompt efectivo

Listing 1: Prompt de ejemplo

```
Error en Model al calcular LTV:

Traceback (most recent call last):
  File "/.../mainMD.py", line 78, in <module>
    cohort_table = cohort_engine.build_cohort_table()
  File "/.../cohort_analyzer.py", line 142, in build_cohort_table
    period_rel = self._calculate_period_diff(first_date, entry['date'])
  File "/.../cohort_analyzer.py", line 98, in _calculate_period_diff
    return end_val - start_val
TypeError: unsupported operand type(s) for -: 'NoneType' and 'int'

Contexto:
- Pa s GT, granularidad mensual, an lisis por categor a.
- DataRepository se ejecut  justo antes y gener  CSV.
- El archivo Resultado_Unit_Economics_GT.csv tiene columna 'cohort' con
    valores como '2024-01', '2024-02', etc.
```

```
- El error ocurre al procesar la cohorte '2024-01' (parece que
   first_date es None para alg n cliente).

  Posible   causa?   Cmo   solucionarlo?
```

10.3 Consideraciones de seguridad al compartir información

- **Nunca** incluir credenciales reales (usuario, contraseña, host) en prompts públicos.
- **Ofuscar** datos sensibles de clientes (IDs, nombres, emails).
- Si se comparten logs, eliminar tokens, llaves API, y cualquier información que pueda identificar a personas.
- Para depuración, usar datos sintéticos (generar CSV de prueba con pocas filas).

11 Soporte y contacto técnico

- **Correo electrónico:** guichocordon@gmail.com
- **Teléfono (WhatsApp):** +502 5364-8127
- **Documentación técnica extendida:**Documentación LTV (más de 400 páginas).
- **Repositorio de código:** ltv-analytics-pacifiko

12 Anexos: Ejemplos de configuración rápida

12.1 Ejemplo de archivo SUPUESTOS.xlsx para Guatemala (hoja `1PGT`)

```
cohort shipping_cost shipping_revenue credit_card_payment cogs retention cac
Q1 -2.5 0 0.03 0.0 0.0 0.0
Q2 -2.5 0 0.03 0.0 0.0 0.0
Q3 -2.5 0 0.03 0.0 0.0 0.0
...
```

12.2 Ejemplo de archivo CAC.xlsx (hoja `GT`)

```
cohort cac
2020-Q1 10.0
2020-Q2 11.5
2021-Q1 12.0
...
```

12.3 Ejemplo de comando SSH para túnel (Linux/Mac)

```
ssh -N -L 3336:localhost:3306 usuario@servidor.com -i ~/.ssh/id_rsa
```